

Reinforcement Learning in Continuous Action Spaces through Sequential Monte Carlo Methods

Final Report

Junyuan Hsieh

August 23, 2026

Outline

- 1 Motivation
- 2 Paper Overview
- 3 Algorithm
- 4 Boat Experiment
- 5 Demo Results
- 6 Extension
- 7 Conclusion

Motivation

- Many real control problems have continuous actions, such as steering angle, acceleration, or force.
- A simple solution is to discretize the action space, but this creates a trade-off:
 - coarse actions: fast learning but poor control precision;
 - fine actions: better control but many actions must be explored.
- The paper focuses on a key question:

Main question

Can we learn good continuous actions without exhaustively discretizing the whole action space?

- Paper: *Reinforcement Learning in Continuous Action Spaces through Sequential Monte Carlo Methods*.
- Main contribution: an actor-critic method called **SMC-learning**.
- The actor represents the policy using a finite set of action samples instead of a parametric distribution.
- The critic estimates action values and guides the actor to move samples toward better actions.

Key idea

Use Monte Carlo samples to represent and update a continuous-action policy.

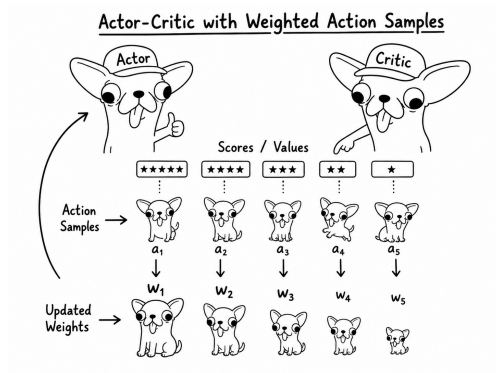
Actor-Critic Structure

- **Actor**: stores a stochastic policy over actions.
- **Critic**: estimates action values $Q(s, a)$.
- For each state s , the actor keeps N action samples:

$$A(s) = \{a_1, a_2, \dots, a_N\}.$$

- Each sample has an importance weight:

$$W(s) = \{w_1, w_2, \dots, w_N\}.$$



Monte Carlo View: Why SMC?

- Monte Carlo methods approximate a complicated distribution using random samples.
- Sequential Monte Carlo goes one step further: the sample set is updated over time.
- In this paper, the target distribution is the policy over continuous actions.

Interpretation

At the beginning, samples explore the action space broadly. During learning, samples with better estimated utility receive larger weights and are duplicated or moved toward promising regions.

- This doesn't need to assume linear or smooth for $Q(s, a)$.
- It also avoids evaluating all possible continuous actions.

SMC-Learning: Update Steps

For each time step:

- 1 **Action selection**: choose one sample action a_i with probability w_i .
- 2 **Critic update**: update $Q(s, a_i)$ using a TD/SARSA-style update.
- 3 **Actor update**: update the sample weights using the change in Q :

$$w_i^{t+1} = \frac{w_i^t \exp(\Delta Q_{t+1}(s, a_i)/\tau)}{\sum_j w_j^t \exp(\Delta Q_{t+1}(s, a_j)/\tau)}.$$

- 4 **Resampling**: if weights become too concentrated, resample actions according to weights.
- 5 **Moving step**: move resampled actions locally using uniform kernels.

Resampling and Moving: The Statistical Computation Part

- Weight degeneracy occurs when only a few samples dominate the policy.
- The paper uses effective sample size:

$$N_{\text{eff}}(s) = \frac{1}{\sum_i w_i^2}.$$

- If $N_{\text{eff}}/N < \sigma$, the action samples are resampled.
- After resampling, samples inherit their parents' action values.
- The moving step perturbs samples locally, so the actor can still search in the continuous action space.

Moving Step: How Do Samples Move?

- After resampling, many samples may become identical.
- This problem is called **sample impoverishment**.
- To avoid this, SMC-learning moves each sample locally.
- Suppose the action samples are sorted:

$$a_1 < a_2 < \cdots < a_N.$$

- For an interior sample a_i , the new action is sampled from

$$a_i^{\text{new}} \sim \text{Uniform} \left(\frac{a_{i-1} + a_i}{2}, \frac{a_i + a_{i+1}}{2} \right).$$

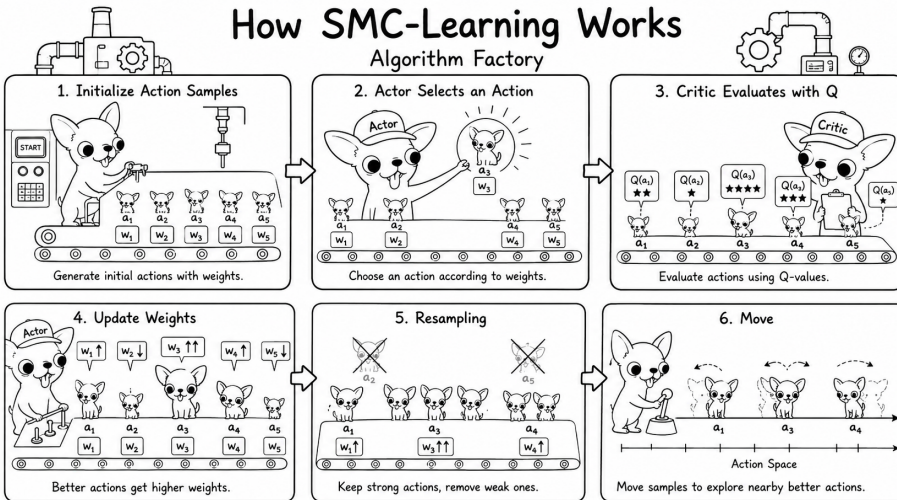
- Therefore, the sample only moves inside a local interval determined by its neighbors.

Intuition

Moving keeps the action space continuous, so SMC-learning is not limited to fixed actions.

How SMC-Learning Works

Algorithm Factory



The Boat Problem

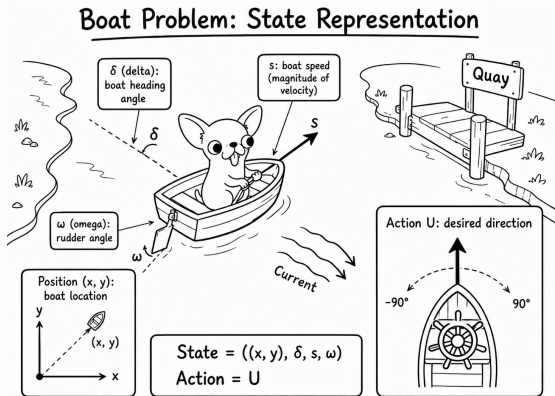
- Task: steer a boat from the left bank to a quay on the right bank.
- State includes position, angle, speed, and angular velocity:

$$(x, y, \delta, s, \Omega).$$

- Action: desired direction

$$U \in [-90^\circ, 90^\circ].$$

- The environment has a strong nonlinear current.



Boat Dynamics: Position Update

Boat position

$$x_{t+1} = \min(200, \max(0, x_t + s_{t+1} \cos(\delta_{t+1})))$$

$$y_{t+1} = \min(200, \max(0, y_t - s_{t+1} \sin(\delta_{t+1}) - E(x_{t+1})))$$

Nonlinear river current

$$E(x) = f_c \left(\frac{x}{50} - \left(\frac{x}{100} \right)^2 \right)$$

- The boat is restricted to the river region:

$$0 \leq x_t, y_t \leq 200.$$

- The horizontal movement depends on speed and heading angle.
- The vertical movement is affected by both heading angle and current $E(x)$.

Internal boat dynamics

$$\delta_{t+1} = \delta_t + I\Omega_{t+1}$$

$$\Omega_{t+1} = \Omega_t + (\omega_{t+1} - \Omega_t) \left(\frac{s_{t+1}}{s_{\max}} \right)$$

$$s_{t+1} = s_t + (s_D - s_t)I$$

$$\omega_{t+1} = \min(\max(p(U_{t+1} - \delta_t), -45^\circ), 45^\circ)$$

Boat Problem: Reward Structure

Reward function

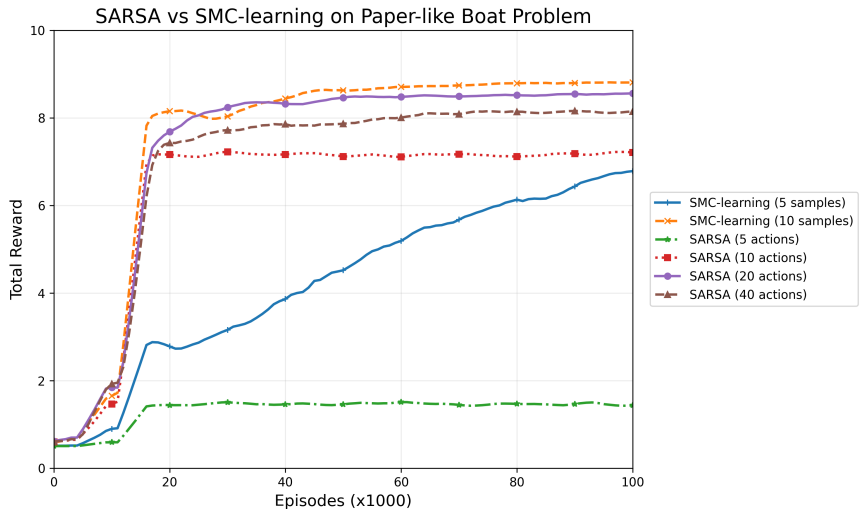
$$R(x, y) = \begin{cases} +10, & (x, y) \in Z_s, \\ D(x, y), & (x, y) \in Z_v, \\ -10, & (x, y) \in Z_f, \\ 0, & \text{otherwise.} \end{cases}$$

My Demo: Reproduction Setting

- I implemented the boat dynamics and SMC-learning mechanism in Python.
- Some details are not fully specified in the paper, such as exact starting points, the exact formula of $D(x, y)$, and plotting/averaging procedures.
- Therefore, I focus on reproducing the qualitative idea rather than exactly matching every numerical curve.

Demo goal

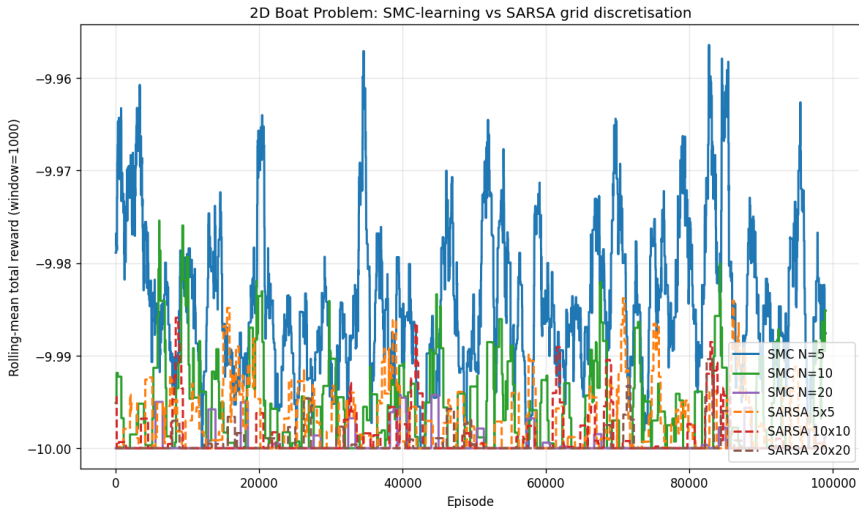
Show that SMC action samples can move toward more rewarding steering directions, while fixed action discretization is less flexible.



Extension: Increasing the Action Dimension

- I also tested a harder setting by increasing the action dimension.
- The performance became much worse and learning was unstable.
- This suggests that although SMC-learning is useful for continuous actions, sample efficiency becomes more challenging in higher-dimensional action spaces.

Extension: Increasing the Action Dimension



- The paper proposes SMC-learning, an actor-critic method for continuous action spaces.
- The actor represents the policy using weighted action samples.
- Importance weighting, resampling, and moving steps allow samples to concentrate near high-value actions.
- In the boat problem, SMC-learning can outperform fixed action discretization using fewer action samples.
- My demo supports the main qualitative idea, but also shows that the result is sensitive to reward design and action dimension.

